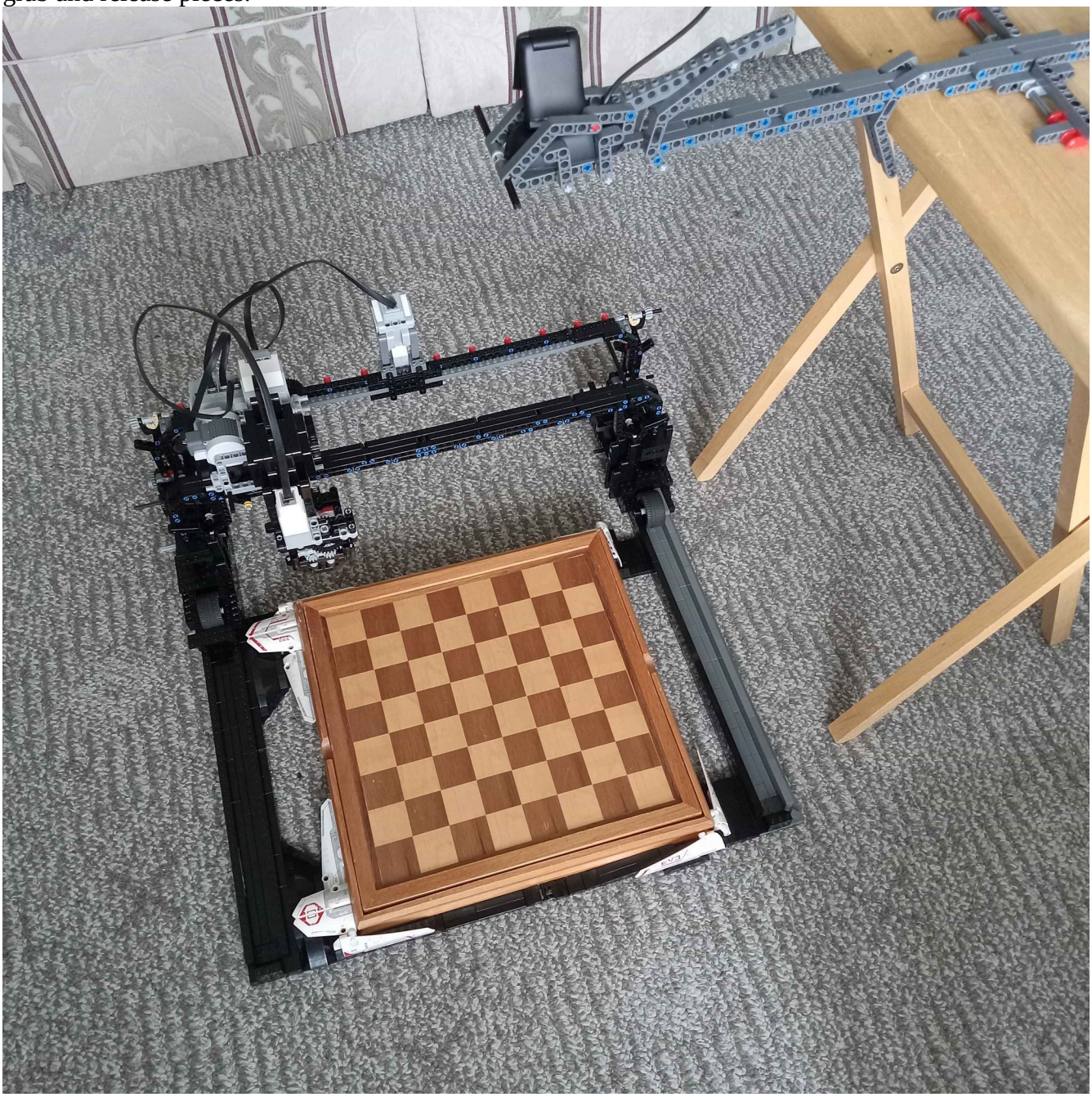
4P78 - EV3 Chess Robot

Devaan Zastrow, AJ Alfonso

This project is an interactive chess-playing robot built using a LEGO Mindstorms EV3 system and powered by Python code. The robot is capable of detecting a human player's move using a webcam, digitally interpreting the board state, using a chess engine (we used Stockfish) to determine the best response, and physically moving chess pieces on the board via a motorized claw system.

Mechanically, the system operates much like a Cartesian gantry (similar to a 3d printer). Two rails provide X and Y movement across the board, and a vertical (Z) mechanism raises and lowers a claw to grab and release pieces.



Hardware used

- LEGO Mindstorms EV3 brick
- 3x Large Mindstorms motors (X, Y, Z movement)
- 1x Medium Mindstorms motor (claw)
- Structural components to construct the rails, a base for them to move along, a rack-and-pinion system for the claw, a claw (In our case, everything was built using LEGO)
- A standard chessboard and pieces
- A webcam (mounted with a clear, steady view of the board)
- A laptop (to run the software)

Software used

- Python
- ev3dev (to allow the EV3 brick to run python programs)
- socket networking (for the main program to communicate with the EV3)
- OpenCV (for image processing)
- python-chess (for board state representation and enforcing rules)
- Stockfish (or any suitable chess engine, for determining moves)
- Pillow & CairoSVG (imaging libraries for drawing a visual representation of the board state)

Software Process Overview

The system can be divided into three main sub-systems, which follow a fairly straightforward code loop:

- Vision
  - Before beginning the game, take an initial image through the webcam and allow the user to mark the location of the board, to determine the locations of each square
  - Capture an image of the board before and after a player moves
  - Divide the image into the squares of the board based on the calibrated shape
  - Compare corresponding squares' pixel values between images to determine those that have large differences
- Chess logic
  - Maintain an internal board state of the game
  - Using the squares that had the largest differences, infer the player's move
  - Verify the validity of the player's move, disallowing any that aren't legal
  - Pass information to the Stockfish engine and query it for a next move
  - Send the engine's chosen move to the EV3 to be applied physically
  - Update the internal board state
- Robot controller
  - Receive an origin, a destination, and a command (either CAPTURE or MOVE)
  - If capturing a piece, first move to the destination square and move the piece located there off of the board
  - Move to origin square
  - Pick up the piece in that square
  - Move to destination square
  - Place carried piece
  - Return to "home" position

Challenges

- The rails, being made out of LEGO, had a lot to be desired in the realm of stability. It functions well enough, but it has enough wiggle room to make positioning a nightmare and the Y-rail tilts slightly every time the claw raises/lowers.
- The EV3 brick has a hard limit of 4 motors; the other 4 ports are dedicated for sensors. This meant that only one motor could be used to power each rail, rather than using two along the base to provide more torque as originally intended. It was assumed that all 8 ports could be used for motors, so this discovery necessitated a partial rebuild of the frame and made the rail even more fragile.
- Communicating with the EV3 brick in order to control the motors programmatically was not directly achievable from the computer, as it can only run programs through the brick itself. This was solved by creating a "server" program to run on the brick in the background, and the "client" main program on the computer would send commands over the socket connection to the brick to be executed.
- The robot's movements had to be incredibly precise; small errors could lead to failed grabs and knocked-over pieces. Much trial and error was required to determine good positions, and even then it can still cause problems if pieces aren't centered well enough.
- Move inference is somewhat sensitive to shadows or differences in lighting, as they create more squares that are seen as "changed" between images. This issue was, at times, amplified by the glare of the board itself. Using a less reflective board or putting some sort of translucent matte sheet over top could help, as could mounting a light source alongside the webcam to provide consistent lighting.

Overall, there's still room for improvement, but this implementation went quite well.